

LearntriX Field Manual

Everything needed to run the platform locally, understand how a request travels through it, and know which parts are finished and which are still scaffolding.

BACKEND Spring Boot 3.4.13 · Java 25	FRONTEND React 19 · TanStack Start
DATABASE PostgreSQL · Flyway v28	SURFACE AREA 166 endpoints · 67 routes

CONTENTS

- Starting the app
- Ports
- Troubleshooting
- Architecture
- Request lifecycle
- Auth & sessions
- Roles
- Onboarding email
- Video storage
- Module map
- Build & deploy
- Known gaps

Starting the app

The platform is two processes: a Spring Boot API and a Vite/TanStack Start frontend. They talk over HTTP and are started independently. PostgreSQL must already be running.

01 Check your toolchain

Java 25 is not optional — `pom.xml` pins `<java.version>25</java.version>` and the build fails on anything older.

VERIFY

```
java --version    # expect 25.x
mvn --version     # expect 3.9+
node --version    # expect 20+
psql --version
```

02 Configure the backend environment

Spring Boot does not read `.env` files by itself. Copy the template and fill in your database password; every value is read through an environment variable, so nothing is hard-coded in the repo.

BACKEND/.ENV

```
DB_URL=jdbc:postgresql://localhost:5432/learntrix
DB_USERNAME=postgres
DB_PASSWORD=your-password
JWT_SECRET=at-least-32-characters-long
APP_CLIENT_URL=http://localhost:8080
# leave MAIL_HOST empty to disable outbound email entirely
MAIL_HOST=smtp.gmail.com
MAIL_USERNAME=you@gmail.com
MAIL_PASSWORD=16-char-app-password
```

EMAIL `MAIL_PASSWORD` must be a 16-character **App Password**, not your account password. A normal password is rejected with `535-5.7.8 Username and Password not accepted`.

03 Create the database

Create an empty database only. Do not build the schema by hand — Flyway owns it and applies all 28 migrations on first boot.

```
PSQL  
  
createdb -U postgres learntrix
```

04 Start the backend

Use the launcher rather than bare Maven. It loads `backend/.env` into the process and stops with a clear message if `DB_PASSWORD` is missing, instead of letting Postgres reject a placeholder.

```
WINDOWS · POWERSHELL
```

```
cd backend  
.\run.ps1
```

```
MACOS · LINUX
```

```
cd backend  
./run.sh
```

A healthy start ends with `Started EdtechApplication in ~19 seconds` and `Tomcat started on port 8081`. Confirm with `curl localhost:8081/actuator/health` — it should return `{"status":"UP"}`.

05 Start the frontend

In a second terminal, from the repository root.

```
REPO ROOT  
  
npm install # first time only  
npm run dev
```

06 Sign in

Migration `v22` seeds a super-admin account at `super@learntrix.com`. The migration stores only a bcrypt hash, so if the password is not already known to you, use **Forgot password** on the login screen and read the reset link out of the backend log — it is printed there on every send, whether or not SMTP is configured.

Ports

PORT	PROCESS	NOTES
8081	Spring Boot API	Fixed in <code>application.yml</code> . Not environment-overridable.
8080	Vite dev server	The usual dev port; CORS also permits 5173, 3000 and 4173.
5432	PostgreSQL	Must be running before the backend starts.

Troubleshooting

“Port 8081 was already in use”

By far the most common failure, and it is not a code problem. The log will show the database connecting, Flyway validating 28 migrations and Hibernate initialising — then failing at the very last step, binding the port. It means another backend instance is still alive.

WINDOWS

```
netstat -ano | findstr :8081
taskkill /PID <pid> /F
```

MACOS • LINUX

```
lsof -i :8081
kill -9 <pid>
```

Activation emails are not arriving

Check `GET /api/admin/mail/status` as an admin. It reports whether SMTP is actually usable and returns a specific hint when it is not. The system is deliberately honest

here: an account is still created when mail fails, and the API says so rather than implying delivery.

The UI shows data that isn't in your database

You are seeing mock data. `VITE_USE MOCKS` must be `false` in the root `.env`. If the variable is missing entirely it **defaults to true**, so an environment without the file silently serves fixtures.

Browser console shows CORS errors

The frontend is on a port the backend doesn't trust. Add it to `CORS_ALLOWED_ORIGINS` in `backend/.env` and restart the API.

Architecture

A conventional layered Spring Boot service behind a single-page React app. There is no message bus or cache on any request path today — Redis and Kafka appear in `docker-compose.yml` and the dependency list, but no application code uses them yet, and their health indicators are deliberately switched off so they cannot drag the service status down.

Frontend

React 19 with TanStack Start (SSR-capable) and file-based routing — 67 route files under `src/routes`. Server state is TanStack Query; UI state is local. Tailwind 4 for styling.

API

21 controllers exposing 166 endpoints under `/api`, documented at `/api-docs` and browsable at `/swagger-ui.html`.

Domain

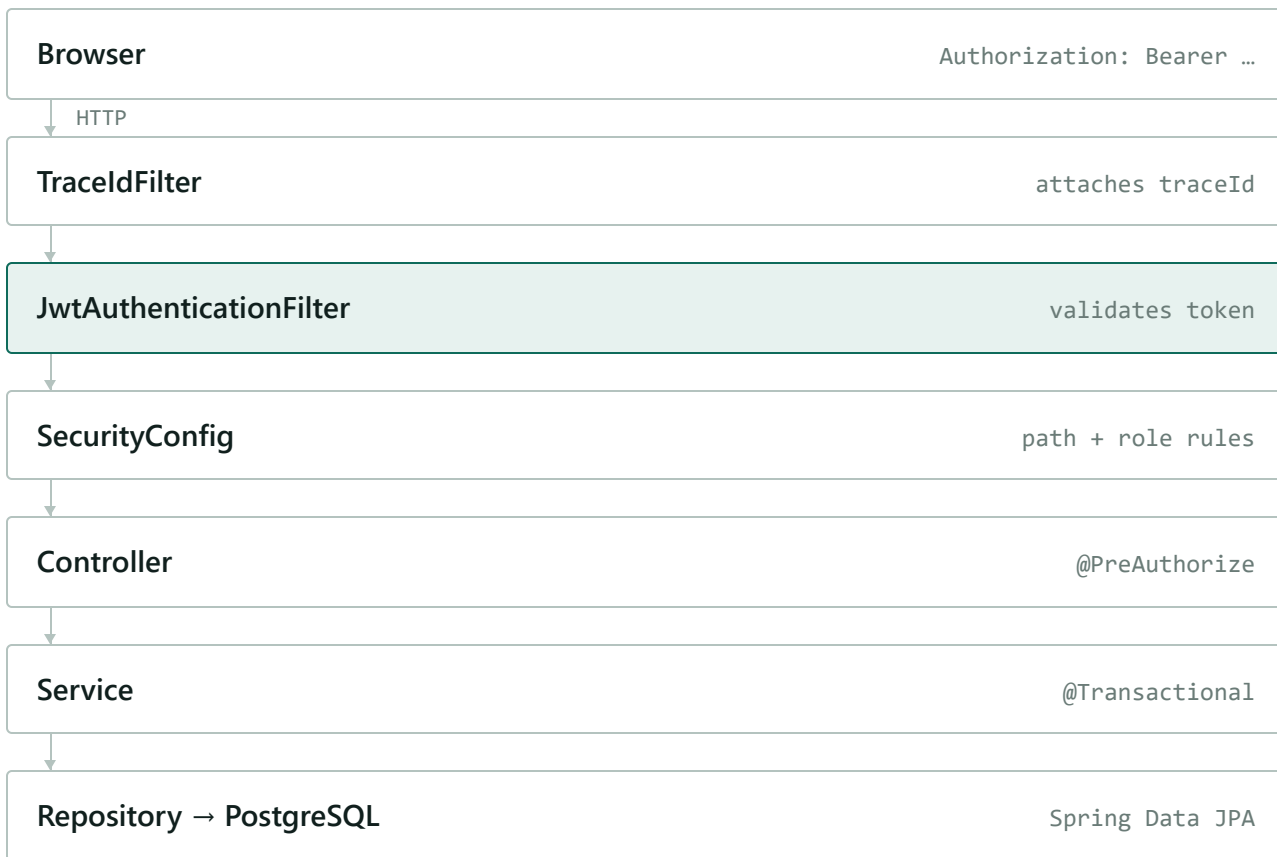
16 services holding business logic, 34 Spring Data JPA repositories, 35 JPA entities.

Schema

PostgreSQL, owned entirely by Flyway. Hibernate runs with `ddl-auto: validate`, so it never alters tables — it only checks that entities match the migrated schema.

Request lifecycle

Every authenticated API call passes through the same chain. The two custom filters run before Spring Security's own.



Responses are uniformly wrapped. Failures are caught by a global handler and returned as structured JSON with a stable error code and the request's `traceId` — never a stack trace.

ERROR SHAPE

```
{
  "success": false,
  "error": { "code": "RESOURCE_NOT_FOUND",
            "message": "Course not found with slug : 'x'" },
  "traceId": "1ab52f09-b095-4c86-b176-bfaeeac016e6"
}
```

Auth & sessions

Stateless JWT. The frontend keeps the session in `localStorage` and re-validates it against `/api/auth/profile` on every app start — that call is what proves a token is still good.

TOKEN	LIFETIME	ENVIRONMENT VARIABLE
Access	15 minutes	JWT_ACCESS_EXPIRATION

TOKEN	LIFETIME	ENVIRONMENT VARIABLE
Refresh	7 days	JWT_REFRESH_EXPIRATION
Remember me	30 days	JWT_REMEMBER_ME_EXPIRATION

Endpoints reachable without a token

Everything else requires authentication. Note that `/api/auth/profile` and `/api/auth/logout` are deliberately *not* public, so a dead token produces a 401 the client can act on rather than a confusing 404.

- `POST /api/auth/login`, `register`, `refresh`
- `POST /api/auth/forgot-password`, `reset-password`, `activate`, `verify-email`
- `GET /api/courses/**` — the public catalogue
- `GET /api/categories`, `/api/testimonials`, `/api/stats`
- `POST /api/enquiries` — the public contact form
- `GET /actuator/health`, `/actuator/info` — for load balancers

NOTE `/actuator/metrics`, `/prometheus`, `/env` and `/beans` all require authentication. Only health and info are open.

Roles

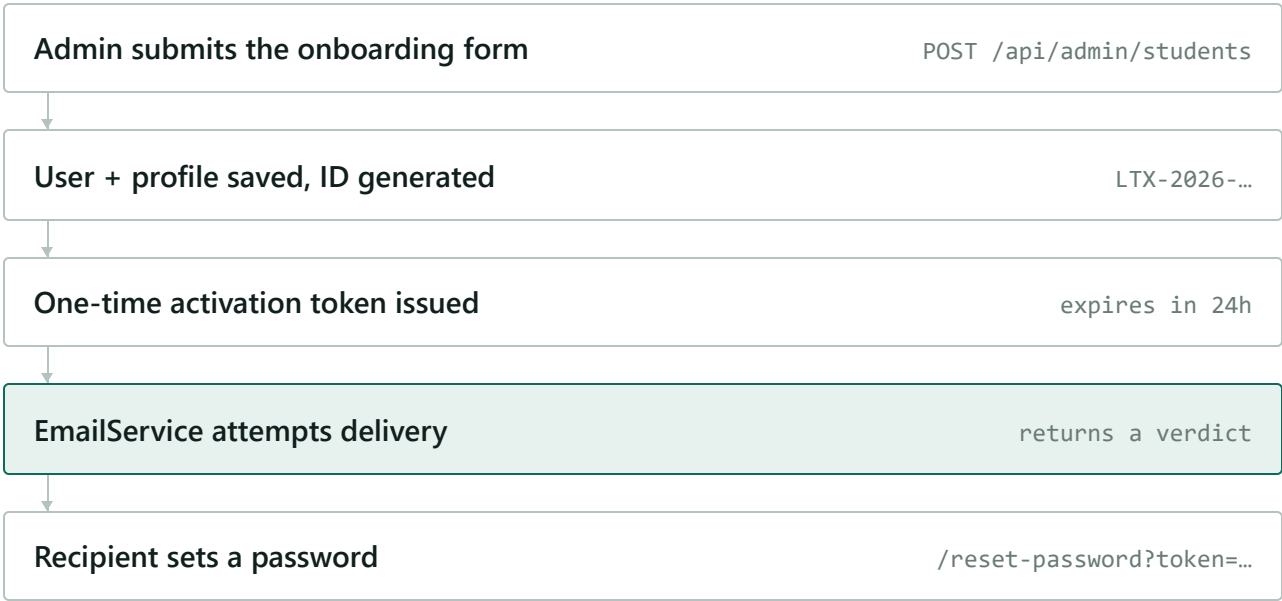
Twelve roles are defined, assigned through a join table and enforced with `@PreAuthorize` at the controller. Permissions themselves are editable at runtime through the RBAC screens rather than being compiled in.

ROLE	PRIMARY SURFACE
super_admin	Everything, including RBAC and staff management
admin	The admin control centre — students, trainers, courses, CRM
teacher	Live classes, recordings, assigned students
student	Courses, recordings, career tools, leaderboard
mentor	Assigned mentees, sessions, notes, alerts
placement_officer	Drives, applications, interviews, offers

ROLE	PRIMARY SURFACE
counsellor	CRM leads and enquiries
content_manager	CMS, articles, course catalogue
reviewer	Content review workflows
support_agent · support	Support queues
corporate	Corporate/partner surface

Onboarding & the activation email

Admins create students and trainers; the platform does not ask them to self-register. That makes the activation email a load-bearing part of the flow, and it is designed around the fact that **the account is created even when the email cannot be delivered**.



The send returns one of three honest states, and the admin UI keys its message off that rather than off the HTTP status:

STATUS	MEANING	WHAT THE ADMIN SEES
SENT	The SMTP server accepted the message.	Success confirmation naming the address.
FAILED	SMTP rejected it or timed out.	Error with the reason; the activation link is shown and copied to the clipboard.

STATUS	MEANING	WHAT THE ADMIN SEES
NOT_CONFIGURED	No SMTP host is set, so nothing was attempted.	Same as failed, plus a pointer to fix the mail settings.

USEFUL The activation link is always written to the backend log, even on success. If email is not configured at all, that log line is how you finish onboarding someone — or use **Resend Invite**, which mints a fresh token and invalidates the old one.

Video storage

Class recordings go through a `VideoStorageService` interface with two implementations. `VideoStorageConfig` picks one at startup from a single property.

LEARNTRIX.VIDEO.STORAGE	IMPLEMENTATION	BEHAVIOUR
local	LocalVideoStorageService	Writes to <code>backend/uploads</code> , served from <code>/uploads/**</code> . The default.
s3	S3VideoStorageService	Issues presigned S3 PUT URLs; public URLs built from the CloudFront domain.

CONTAINERS A container filesystem is ephemeral. Running with `local` storage in ECS or any container platform loses every upload on redeploy — the `prod` profile therefore defaults this to `s3`.

Module map

Where to look when changing a feature.

AREA	FRONTEND	CONTROLLER
Admin control centre	<code>admin.*</code>	<code>AdminController</code>
Auth & activation	<code>login</code> , <code>register</code> , <code>reset-password</code>	<code>AuthController</code>
Course catalogue	<code>courses.*</code> , <code>admin.courses</code>	<code>CourseController</code>

AREA	FRONTEND	CONTROLLER
Live classes	teacher.live-classes, student.live-classes	LiveClassController
Recordings	*.recordings.*	ClassRecording · Admin · StudentRecording
Career & placement	student.career.*, admin.career.*	PlacementController · JobController
CRM & enquiries	admin.crm.*	CrmController · EnquiryController
Mentorship	—	MentorController
Assessments	—	QuizController · AssignmentController
Roles & staff	admin.settings	RbacController

Building

PRODUCTION BUILD

```
# backend → backend/target/edtech-backend-0.0.1-SNAPSHOT.jar
cd backend && mvn clean package

# frontend → .output/
npm run build
```

Profiles

`dev` is the default and carries working defaults for everything. `prod` deliberately removes those defaults so a misconfigured deployment fails at startup instead of running against the wrong database.

VARIABLE	REQUIRED IN PROD	PURPOSE
DB_URL · DB_USERNAME · DB_PASSWORD	yes	RDS connection
JWT_SECRET	yes	Token signing key, 32+ chars
AWS_S3_BUCKET	yes	Recording storage bucket
AWS_S3_REGION	no	Defaults to <code>us-east-1</code>
AWS_CLOUDFRONT_DOMAIN	no	Public video URL host
CORS_ALLOWED_ORIGINS	effectively	Production frontend origin
APP_CLIENT_URL	effectively	Origin used to build links inside emails
MAIL_HOST · MAIL_USERNAME · MAIL_PASSWORD	no	Empty host disables sending entirely

BUILD-TIME The frontend's API URL is **baked in when you build**, not read at runtime. A build without `VITE_API_URL` and `VITE_USE MOCKS=false` ships a bundle pointing at `localhost`, or serving mock data. Set both in the build step.

AWS On ECS, attach an IAM task role rather than setting `AWS_ACCESS_KEY_ID` / `AWS_SECRET_ACCESS_KEY`. The SDK's default credentials provider picks the role up automatically and nothing long-lived sits in the task definition. Point the load balancer health check at `/actuator/health`.

Known gaps

These look finished in the UI but are not. Worth knowing before demonstrating the platform to anyone.

“Assign” on the Trainers page

Shows a toast reading “Assignment editor opens the batch planner.” There is no batch planner — that string appears exactly once in the codebase. The button does nothing else.

Use instead: assign a trainer from the student’s detail page, where course, trainer and batch are allocated together. That path is fully wired.

Create-batch form

Renders fields for name, course, trainer and dates, then submits an empty object. It reports “Batch submitted.” and silently discards everything typed. The backend endpoint is real; the form never builds a payload.

`/upload-local` endpoints

Marked “during development” but live in every profile. They bypass the storage abstraction and write straight to container disk, so they should be gated before a container deployment.

Redis & Kafka

Present as dependencies and compose services, used by nothing. Repository scanning and health checks are switched off for both so they cannot affect startup or reported status.

Compiled from the codebase at its current state — endpoint counts, token lifetimes, migration version and security rules were read from source rather than documentation, so they reflect what actually runs.